EventHub
CONNECT. PARTICIPATE. EXPERIENCE.

PROJETO ACADÊMICO | UECE

Relatório Final do Projeto EventHub

Plataforma de gerenciamento de eventos acadêmicos e profissionais

ARQUITETURA

Microsserviços com comunicação síncrona e assíncrona

NOTA DE ESCOPO

Este documento apresenta o cenário de entrega final solicitado, considerando concluído o escopo definido: cinco microsserviços implementados e três serviços adicionais modelados. Descreve a solução final sem atribuir resultados de testes, métricas de desempenho ou publicação pública que não estejam comprovados.

Universidade Estadual do Ceará - UECE

Relatório técnico | 2 de novembro de 2026

Sumário

1. Visão geral do projeto . . . 3
2. Problema que o projeto resolve . . . 3
3. Objetivos . . . 3
4. Público-alvo . . . 4
5. Requisitos funcionais . . . 5
6. Requisitos não funcionais . . . 6
7. Histórico de decisões e evolução . . . 6
8. Tecnologias utilizadas . . . 7
9. Estrutura de diretórios . . . 8
10. Arquitetura do sistema . . . 10
11. Descrição detalhada dos módulos . . . 12
12. Funcionamento das funcionalidades . . . 16
13. Regras de negócio . . . 19
14. Modelagem de dados . . . 21
15. Fluxos de execução . . . 25
16. Integrações externas . . . 27
17. Configuração e execução . . . 28
18. Testes . . . 30
19. Estado final do desenvolvimento . . . 31
20. Limitações e melhorias futuras . . . 32
21. Comparação entre planejamento e entrega . . . 33
22. Encerramento e próximos passos recomendados . . . 34

1. Visão geral do projeto

O EventHub foi desenvolvido como uma plataforma web para organizar eventos acadêmicos e profissionais, conectando participantes, organizadores e instituições em um ambiente integrado.

A plataforma reúne cadastro e autenticação, gerenciamento de organizações, publicação de eventos, programação de atividades, inscrições, reservas de vagas, pagamentos simulados e notificações.

O projeto adotou uma arquitetura baseada em microsserviços para demonstrar separação de responsabilidades, propriedade dos dados, interoperabilidade entre tecnologias e tratamento de falhas em operações distribuídas.

O escopo da entrega foi organizado em dois grupos:

Grupo	Serviços
Implementação do protótipo	Identity, Event, Registration, Payment e Notification
Modelagem arquitetural	Attendance, Certificate e Reporting

A modelagem dos três últimos serviços faz parte da entrega arquitetural. Sua implementação operacional foi reservada para evolução posterior, conforme o recorte original do trabalho.

2. Problema que o projeto resolve

O gerenciamento de um evento envolve operações relacionadas, mas com regras diferentes: identificar usuários, organizar atividades, controlar vagas, acompanhar pagamentos e comunicar resultados.

Sem uma separação adequada dessas responsabilidades, alterações em uma funcionalidade podem comprometer outras partes do sistema. Operações concorrentes também exigem cuidados específicos: duas pessoas podem tentar reservar a última vaga ao mesmo tempo, um pagamento pode chegar depois da expiração da reserva e uma notificação pode falhar após a confirmação da inscrição.

O EventHub trata esse problema por meio de uma plataforma integrada, na qual cada domínio possui regras e persistência próprias, mantendo a coordenação necessária para o funcionamento do fluxo completo.

Do ponto de vista dos usuários, o sistema oferece:

- Um ambiente para localizar eventos e consultar suas atividades.
- Inscrições com estados claros e controle de disponibilidade.
- Uma área institucional para administrar eventos e equipes.
- Acompanhamento das reservas e dos pagamentos simulados.

Do ponto de vista arquitetural, o projeto demonstra como essas funcionalidades podem operar sem compartilhar diretamente as tabelas de seus bancos.

3. Objetivos

O objetivo principal foi construir um protótipo web integrado de gerenciamento de eventos, utilizando microsserviços e pelo menos duas tecnologias de implementação.

Os objetivos específicos foram:

1. Implementar autenticação e autorização por usuário e organização.
2. Permitir o cadastro, a publicação e o cancelamento de eventos.
3. Organizar atividades com horários, responsáveis, capacidade e preço.
4. Impedir inscrições acima da capacidade configurada.
5. Diferenciar inscrições gratuitas e pagas.
6. Confirmar inscrições pagas somente após aprovação do pagamento simulado.
7. Liberar vagas após cancelamento, recusa ou expiração.
8. Promover participantes da lista de espera.
9. Processar notificações sem bloquear a confirmação da inscrição.
10. Disponibilizar uma interface baseada no Figma.
11. Executar o ambiente de maneira reproduzível com Docker Compose.
12. Documentar contratos, decisões, fluxos, estados e limitações.

A possibilidade de exposição pela internet foi prevista por meio do Cloudflare Tunnel, mantendo o Nginx como ponto de entrada da aplicação.

4. Público-alvo

O sistema contempla três formas principais de participação:

Público	Utilização
Participante	Consultar eventos, realizar inscrições, acompanhar reservas e pagamentos e manter seu perfil
Responsável institucional	Cadastrar uma organização, administrar seus dados e adicionar organizadores
Organizador	Administrar eventos das organizações às quais está vinculado

A mesma conta pode participar de eventos e integrar equipes organizadoras.

O papel global da conta é `PARTICIPANT`. As permissões institucionais são definidas por vínculo com cada organização:

- `OWNER`: responsável pela organização.
- `ORGANIZER`: integrante da equipe organizadora.

Essa decisão evita que um papel global de organizador conceda acesso indevido aos eventos de outras instituições.

5. Requisitos funcionais

Requisito	Descrição	Atendimento na entrega
RF01	Cadastro, autenticação e perfis	Implementado no Identity
RF02	Cadastro e gerenciamento de eventos	Implementado no Event
RF03	Atividades e programação	Implementado no Event
RF04	Consulta e busca de eventos	Implementado na API e no catálogo
RF05	Inscrições em atividades	Implementado no Registration
RF06	Reservas, cancelamentos e lista de espera	Implementado no Registration
RF07	Pagamentos simulados	Implementado no Payment
RF08	Notificações	Implementado com entrega simulada e RabbitMQ
RF09	Check-in e presença	Modelado no Attendance
RF10	Certificados	Modelado no Certificate
RF11	Relatórios e indicadores	Modelado no Reporting

Também foram incorporados ao desenvolvimento:

- Acessos separados para participante e organização.
- Cadastro institucional com criação conjunta da conta e da organização.
- Recuperação e redefinição de senha com entrega simulada.
- Troca autenticada de senha.
- Sessão persistente de uma hora.
- Menu da conta e confirmação de saída.
- Interface responsiva para desktop e celular.

Pagamentos e notificações foram mantidos como simulações, adequadas ao contexto acadêmico.

6. Requisitos não funcionais

Requisito	Solução arquitetural
Modularidade	Serviços separados por domínio
Evolução independente	Código, dependências e Dockerfile próprios
Interoperabilidade	REST/JSON entre aplicações Node.js e Python
Consistência de vagas	Transações e bloqueios no PostgreSQL
Tratamento de duplicidade	Chaves idempotentes e restrições de unicidade
Resiliência de notificações	Outbox e processamento assíncrono
Observabilidade	Logs estruturados, request ID e endpoints de saúde
Segurança	Hash de senha, JWT, autorização e variáveis de ambiente
Persistência	Bancos próprios e volumes Docker
Reprodutibilidade	Docker Compose e manifestos de dependências
Usabilidade	Interface baseada no Figma e estados explícitos

A separação dos serviços permite planejar escalabilidade seletiva. O protótipo utiliza uma implantação local, sem caracterizar uma infraestrutura de alta disponibilidade.

7. Histórico de decisões e evolução

O projeto evoluiu de uma definição arquitetural inicial para a implementação progressiva do fluxo principal.

Etapa	Decisão ou evolução
Concepção	Plataforma acadêmica de eventos com arquitetura de microsserviços
Definição tecnológica	React no frontend; Node.js e Python no backend
Infraestrutura	Docker Compose, Nginx, PostgreSQL e RabbitMQ
Identidade	Autenticação JWT e credenciais protegidas com scrypt
Organizações	Permissões por vínculo OWNER e ORGANIZER
Acessos	Separação das telas de participante e organização
Senhas	Mínimo definido em oito caracteres
Interface	Adaptação das telas do Figma e uso da imagem original da marca
Cadastro institucional	Criação transacional da conta, organização e vínculo responsável

Etapa	Decisão ou evolução
Eventos	Programação, atividades, publicação e cancelamento
Inscrições	Reservas, concorrência, cancelamento e lista de espera
Pagamentos	Simulação com decisão persistida e conciliação
Mensageria	Outbox, RabbitMQ e deduplicação no consumidor
Sessão	Ampliação para uma hora, com restauração após recarregamento
Revisão visual	Ajustes de espaçamento, filtros, responsividade e modais

As decisões principais foram registradas em cinco ADRs:

ADR	Tema
001	Duas stacks e propriedade dos bancos
002	Autenticação, proteção de credenciais e sessão
003	Organizações e permissões por vínculo
004	Capacidade fixa após publicação e reservas serializadas
005	Conciliação de pagamento e outbox

Os registros estão em `docs/adr/`.

8. Tecnologias utilizadas

Frontend

Tecnologia	Finalidade
React	Construção da interface
TypeScript	Tipagem dos componentes e contratos
Vite	Desenvolvimento e build
React Router	Navegação
Context API	Gerenciamento da sessão
CSS	Estilização e responsividade
Inter	Tipografia local
Playwright	Testes de navegador

Backend

Tecnologia	Aplicação
Node.js e Express	Gateway, Identity e Payment
Python e FastAPI	Event, Registration e Notification
Pydantic	Validação das entradas nos serviços Python
pg	PostgreSQL nos serviços Node
psycopg	PostgreSQL nos serviços Python
jose	Emissão e validação de JWT
express-rate-limit	Limitação de tentativas
pika	Comunicação com RabbitMQ

Infraestrutura

Tecnologia	Finalidade
Docker	Containerização
Docker Compose	Composição do ambiente
Nginx	Proxy reverso e distribuição do frontend
PostgreSQL	Persistência relacional
RabbitMQ	Mensageria assíncrona
Cloudflare Tunnel	Exposição do ambiente pela internet
Git e GitHub	Versionamento

Não foi introduzido ORM. Os serviços utilizam SQL parametrizado, mantendo o acesso aos dados dentro do domínio proprietário.

9. Estrutura de diretórios

A organização principal do projeto é:

```
eventhub/
|-- frontend/
|   |-- public/
|   |   `-- figma/
|   |-- src/
|   |   |-- auth/
|   |   |   |-- access.ts
|   |   |   |-- AuthContext.tsx
|   |   |   `-- session.ts
```

```
|   |   |-- components/
|   |   |   |-- auth/
|   |   |   |-- form/
|   |   |   |-- profile/
|   |   |   |-- AppLayout.tsx
|   |   |   |-- AuthLayout.tsx
|   |   |   |-- Brand.tsx
|   |   |   `-- Business.tsx
|   |   |-- pages/
|   |   |   |-- events/
|   |   |   |-- organization/
|   |   |   |-- participant/
|   |   |   |-- registrations/
|   |   |   |-- AccessPage.tsx
|   |   |   |-- EnvironmentPage.tsx
|   |   |   |-- OrganizationsPage.tsx
|   |   |   |-- ProfilePage.tsx
|   |   |   `-- RecoveryPage.tsx
|   |   |-- services/
|   |   |-- types/
|   |   |-- App.tsx
|   |   |-- main.tsx
|   |   `-- arquivos CSS
|   |-- tests/
|   |-- Dockerfile
|   |-- nginx.conf
|   |-- playwright.config.ts
|   |-- vite.config.ts
|   |-- tsconfig.json
|   |-- package.json
|   `-- package-lock.json
|-- gateway/
|   |-- src/
|   |   |-- config/
|   |   |-- middleware/
|   |   |-- routes/
|   |   |-- security/
|   |   |-- app.js
|   |   `-- server.js
|   |-- test/
|   |-- Dockerfile
|   |-- package.json
|   `-- package-lock.json
|-- services/
|   |-- identity/
|   |   |-- src/
|   |   |   |-- database/
|   |   |   |-- middleware/
|   |   |   |-- organizations/
|   |   |   |-- routes/
|   |   |   |-- security/
|   |   |   `-- users/
|   |   |-- test/
|   |   |-- Dockerfile
|   |   |-- package.json
|   |   `-- package-lock.json
|   |-- events/
|   |   |-- app/
|   |   |   |-- api/
|   |   |   |-- database.py
|   |   |   |-- domain.py
|   |   |   |-- main.py
|   |   |   `-- observability.py
|   |   |-- Dockerfile
```

```
|   |   `-- requirements.txt
|   |-- registrations/
|   |   |-- app/
|   |   |   |-- api/
|   |   |   |-- database.py
|   |   |   |-- domain.py
|   |   |   |-- main.py
|   |   |   `-- observability.py
|   |   |-- Dockerfile
|   |   `-- requirements.txt
|   |-- payments/
|   |   |-- src/
|   |   |   |-- database/
|   |   |   |-- middleware/
|   |   |   |-- routes/
|   |   |   |-- app.js
|   |   |   |-- payments.js
|   |   |   `-- server.js
|   |   |-- test/
|   |   |-- Dockerfile
|   |   |-- package.json
|   |   `-- package-lock.json
|   `-- notifications/
|       |-- app/
|       |   |-- api/
|       |   |-- consumer.py
|       |   |-- database.py
|       |   |-- main.py
|       |   `-- observability.py
|       |-- Dockerfile
|       `-- requirements.txt
|-- infra/
|   |-- db/
|   |   `-- 01-init.sh
|   `-- nginx/
|       `-- nginx.conf
|-- scripts/
|   |-- init-identity-keys.mjs
|   `-- init-internal-key.mjs
|-- tests/
|   `-- integration/
|-- docs/
|   |-- adr/
|   |-- api/
|   |-- architecture/
|   |-- validation/
|   `-- PLAN.md
|-- compose.yaml
|-- .env.example
|-- .gitattributes
|-- .gitignore
`-- README.md
```

Cada aplicação possui gerenciamento próprio de dependências. Os diretórios de dependências instaladas, builds e capturas de testes não fazem parte do código autoral da solução.

10. Arquitetura do sistema

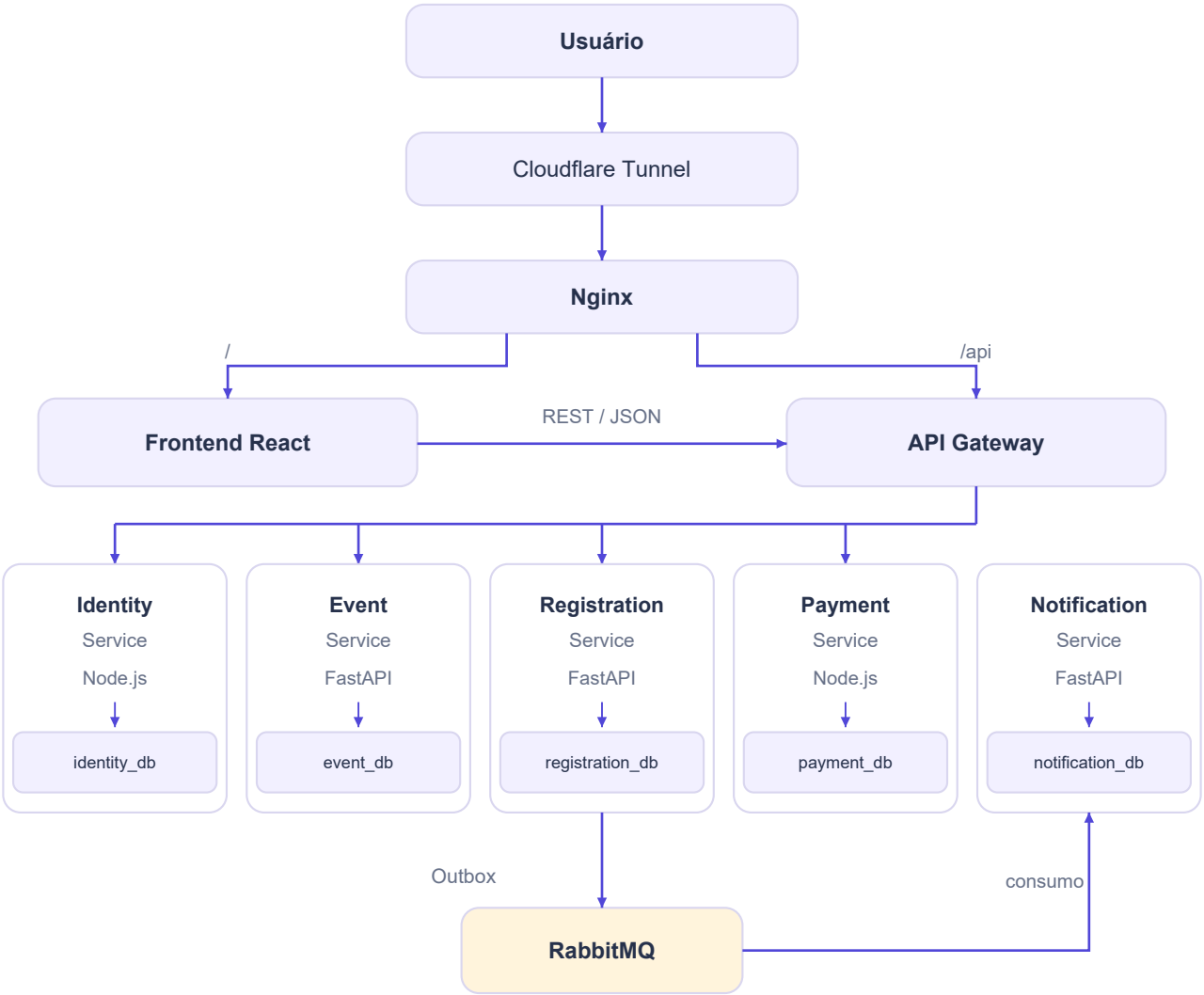
O acesso externo foi organizado segundo o fluxo:

Usuário → Cloudflare Tunnel → Nginx → Frontend / Gateway → Microsserviços

O frontend apresenta as funcionalidades e consome as APIs REST. O Gateway concentra a entrada externa das APIs, valida a autenticação quando necessária e encaminha as requisições.

Os serviços continuam responsáveis pelas permissões e pelas regras de seus próprios domínios.

Figura 1. Arquitetura de acesso, serviços e propriedade dos dados

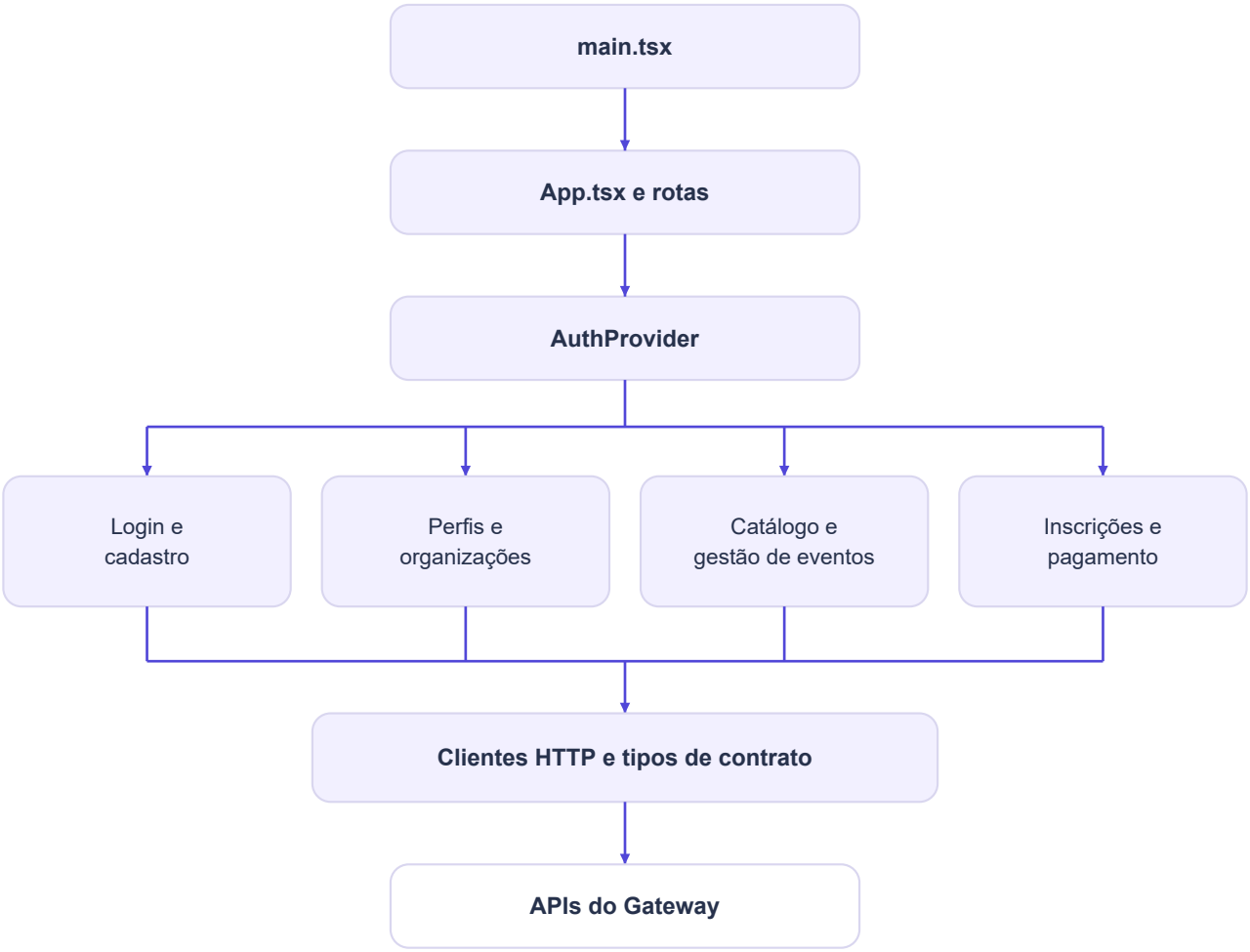


Chamadas internas não precisam passar pelo Gateway. Payment e Registration, por exemplo, utilizam comunicação direta pela rede Docker.

Os cinco bancos lógicos compartilham uma instância PostgreSQL, mas possuem credenciais e permissões específicas.

Organização do frontend

Figura 2. Organização dos módulos do frontend



11. Descrição detalhada dos módulos

11.1 Frontend

O frontend foi organizado em páginas, componentes reutilizáveis, clientes de API e tipos.

Elemento	Responsabilidade
App	Definir as rotas
AuthProvider	Disponibilizar sessão e operações de autenticação
AuthLayout	Estruturar login e cadastro
AppLayout	Navegação, identidade do usuário e saída

Elemento	Responsabilidade
BusinessLayout	Proteger páginas de negócio
Brand	Apresentar a imagem da marca
PasswordInput	Campo com exibição opcional da senha
ProfileSelector	Alternar os cadastros
TermsDialog	Apresentar termos e privacidade
PasswordSection	Executar troca autenticada de senha
EventCard	Apresentar eventos
EventFilters	Oferecer busca e filtros
Feedback	Apresentar carregamento e erros

Os clientes HTTP estão em `frontend/src/services/`. A sessão está concentrada em `frontend/src/auth/`.

11.2 API Gateway

O Gateway centraliza as chamadas externas e não possui banco próprio.

Seus componentes principais são:

- `createApp`: composição Express.
- `createVerifier`: validação dos JWTs.
- `createIdentityRouter`: encaminhamento de identidade e organizações.
- `createBusinessRouter`: encaminhamento de eventos, inscrições e pagamentos.
- `createHealthRouter`: diagnóstico local e agregado.
- `requestLogging`: registro das requisições.

Os callbacks internos de pagamento não são publicados pelas rotas externas.

11.3 Identity Service

O Identity mantém:

- Contas e credenciais.
- Dados de perfil.
- Organizações.
- Vínculos institucionais.
- Recuperação de senha.
- Emissão de tokens.

A implementação separa transporte HTTP, regras, persistência e segurança.

Módulo	Elementos
Usuários	createUserService, createUserRepository
Organizações	createOrganizationService, createOrganizationRepository
Segurança	hashPassword, verifyPassword, createTokens
Recuperação	createRecoveryRouter
Banco	createPool, migrate

O cadastro institucional utiliza transação para criar conta, organização e vínculo `OWNER` como uma única operação.

11.4 Event Service

O Event mantém os eventos e sua programação.

Seus modelos de entrada são:

- `EventInput`.
- `ActivityInput`.
- `Speaker`.

Suas operações principais são:

- `listing`.
- `detail`.
- `save`.
- `create`.
- `update`.
- `transition`.

O serviço valida o vínculo institucional consultando o Identity por API.

A capacidade máxima, o preço e a programação pertencem ao Event. Após a publicação, esses dados permanecem fixos no recorte do protótipo, preservando a consistência com as reservas.

11.5 Registration Service

O Registration mantém a ocupação efetiva das atividades.

Função	Responsabilidade
reserve	Solicitar uma inscrição
lock	Bloquear o inventário da atividade
maintain	Expirar reservas e promover a fila
owned	Restringir acesso ao participante proprietário

Função	Responsabilidade
cancel	Cancelar inscrição
payment	Aplicar resultado do pagamento
event_message	Registrar evento na outbox
worker	Executar manutenção e publicação assíncrona

A disponibilidade não é decidida pelo navegador. A decisão ocorre no banco, dentro de uma transação.

11.6 Payment Service

O Payment registra transações simuladas.

Seus principais elementos são:

- `migratePayments`.
- `createPaymentRouter`.
- `reconcile`.
- `startReconciliation`.
- `view`.

A primeira decisão de uma transação é persistida antes da comunicação com Registration. Isso permite retomar a conciliação se houver indisponibilidade temporária.

O serviço não solicita cartão, conta bancária ou integração financeira real.

11.7 Notification Service

O Notification consome os eventos de inscrição publicados no RabbitMQ.

As funções centrais são:

- `migrate`: inicialização da persistência.
- `consume`: consumo, deduplicação e tratamento das mensagens.
- `db`: acesso ao banco proprietário.

A entrega simulada é representada por um registro persistido. A falha do consumidor não desfaz a inscrição que originou a mensagem.

11.8 Serviços modelados

Serviço	Responsabilidade definida
Attendance	Check-in, presença e prevenção de duplicidade
Certificate	Elegibilidade e emissão após evento e presença válida
Reporting	Relatórios e projeções para organizadores

Esses serviços permanecem representados na arquitetura, sem containers ou endpoints operacionais no protótipo inicial.

12. Funcionamento das funcionalidades

Cadastro e autenticação

O participante informa nome, e-mail e senha, podendo complementar CPF e telefone.

O cadastro institucional inclui os dados da instituição e cria seu vínculo inicial de responsabilidade.

O login retorna JSON:

```
{
  "accessToken": "TOKEN",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user": {}
}
```

A senha é protegida com scrypt e não retorna nas respostas.

Sessão de uma hora

A sessão permanece disponível por uma hora a partir do login.

Token, dados públicos do usuário e prazo absoluto são persistidos no navegador. Atualizar a página ou reabrir o navegador não reinicia a contagem.

Ao sair da conta:

1. A sessão em memória é removida.
2. O armazenamento persistido é limpo.
3. As outras abas da mesma origem recebem a atualização.

A expiração também remove a sessão local. Não há renovação automática por refresh token.

Perfil e senha

O participante pode atualizar seus dados pessoais. O responsável institucional pode editar os dados da organização.

A troca de senha exige a senha atual e confirmação da nova senha. Após a alteração autenticada, a interface encerra a sessão local.

O e-mail institucional é separado da credencial pessoal de login.

Recuperação de senha

O fluxo utiliza token aleatório de uso único, com validade de trinta minutos.

O banco armazena seu hash. A entrega do link é simulada em uma caixa local do ambiente, sem um endpoint público para consultar os arquivos.

O formulário público apresenta resposta genérica, evitando informar diretamente se o endereço está cadastrado.

Eventos e atividades

O organizador:

1. Escolhe uma organização à qual está vinculado.

2. Preenche os dados gerais.
3. Define modalidade, local e período.
4. Adiciona palestrantes.
5. Configura atividades.
6. Salva o rascunho.
7. Publica o evento.

Cada atividade possui capacidade e preço próprios.

Inscrição gratuita

Quando existe vaga, uma atividade de preço zero gera inscrição confirmada diretamente.

Se a atividade estiver lotada, o participante entra na lista de espera.

Inscrição paga

Quando existe vaga, o sistema cria uma reserva temporária em `PENDING_PAYMENT`.

O participante escolhe o resultado da simulação. A aprovação somente confirma a inscrição se a reserva ainda estiver válida.

Recusa, expiração ou cancelamento retiram a inscrição da ocupação efetiva e permitem a promoção da fila.

Páginas e rotas

Página	Rota
Escolha de acesso	/ ou /acesso
Login participante	/participante/login
Cadastro participante	/participante/cadastro
Login organização	/organizacao/login
Cadastro institucional	/organizacao/cadastro
Recuperação	/recuperar-senha
Redefinição	/redefinir-senha
Catálogo	/eventos
Detalhes e atividades	/eventos/:id
Gestão de eventos	/organizacao/eventos
Criação de evento	/organizacao/eventos/novo
Edição e gerenciamento	/organizacao/eventos/:id
Perfil participante	/perfil
Perfil institucional	/organizacao/perfil e /organizacao/perfil/:id
Organizações e equipe	/organizacoes
Reserva de atividade	/atividades/:eventId/:activityId/inscricao
Pagamento e confirmação	/pagamento/:id
Minhas inscrições	/minhas-inscricoes
Diagnóstico	/diagnostico

Os popups compreendem menu da conta, confirmação de saída, termos, privacidade e o diálogo de inscrição/pagamento.

13. Regras de negócio

Regra	Aplicação
RN01	Ocupação limitada pela capacidade, inclusive sob concorrência
RN02	Inscrição paga depende de aprovação dentro do prazo
RN03	Recusa e expiração liberam vaga
RN04	Cancelamento atualiza ocupação e pode promover a fila
RN05	Falha de notificação não desfaz confirmação
RN06	Certificado depende de término do evento e presença válida; regra modelada
RN07	Participante acessa seus dados; organizador administra organizações vinculadas

Controle concorrente

A atividade possui uma linha de inventário no Registration.

Antes de decidir a ocupação, a transação bloqueia essa linha com `SELECT FOR UPDATE`. Requisições simultâneas para a mesma atividade são serializadas nesse ponto.

Ocupam vaga:

- `CONFIRMED`.
- `PENDING_PAYMENT`.

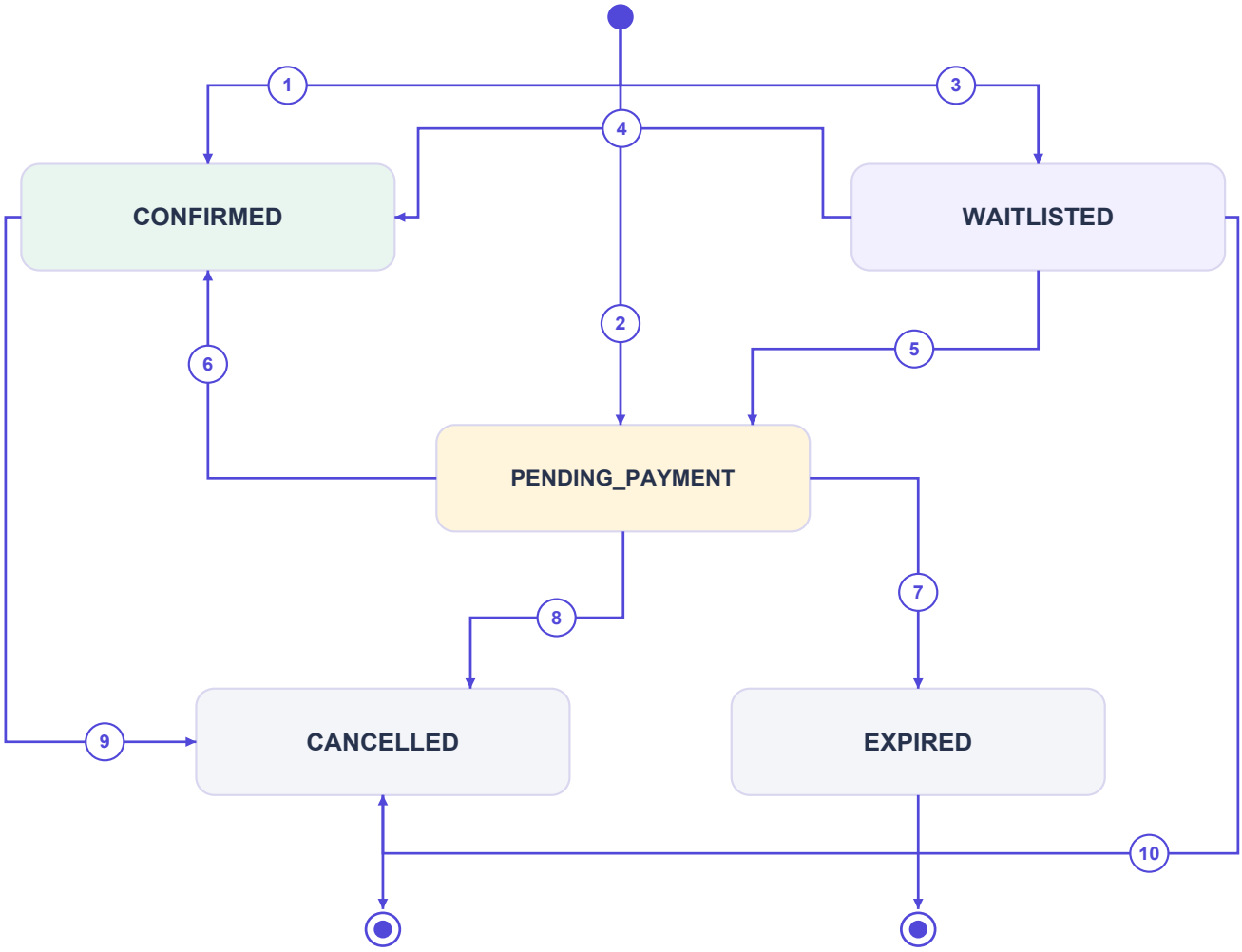
Não ocupam vaga:

- `WAITLISTED`.
- `CANCELLED`.
- `EXPIRED`.

Uma restrição adicional impede duas inscrições ativas do mesmo usuário na mesma atividade.

Estados das inscrições

Figura 3. Estados e transições das inscrições



Transições

1. Gratuita com vaga
2. Paga com vaga
3. Sem vaga
4. Promoção em atividade gratuita
5. Promoção em atividade paga
6. Aprovação dentro do prazo
7. Prazo vencido
8. Recusa ou cancelamento
9. Cancelamento de inscrição confirmada
10. Cancelamento na lista de espera

Ocupam vaga: CONFIRMED e PENDING_PAYMENT.
WAITLISTED, CANCELLED e EXPIRED não ocupam vaga.

A promoção da fila considera a ordem de entrada. Quando a atividade é paga, a promoção inicia um novo prazo para pagamento.

Estados dos pagamentos

Estado	Significado
PENDING	Decisão registrada, aguardando conciliação
APPROVED	Registration confirmou a inscrição
DECLINED	Simulação recusada
EXPIRED	A operação não pode confirmar a reserva por expiração ou estado terminal

Uma aprovação tardia não reabre uma inscrição expirada ou cancelada.

Cancelamento de evento

O Event registra o cancelamento. O Registration concilia esse estado nas operações individuais e no processamento periódico.

Essa coordenação utiliza consistência eventual. Não há uma transação SQL distribuída entre os dois bancos.

14. Modelagem de dados

Propriedade

Banco	Proprietário
identity_db	Identity
event_db	Event
registration_db	Registration
payment_db	Payment
notification_db	Notification

Referências entre bancos são identificadores lógicos. Não são criadas chaves estrangeiras entre serviços.

Identity

Tabela	Dados principais
users	Identificador, nome, e-mail, hash, papel, CPF, telefone, preferências e datas
organizations	Identificador, nome, descrição, CNPJ, razão social, tipo e contatos
organization_members	Organização, usuário, papel e data do vínculo
password_resets	Hash do token, usuário e expiração
schema_migrations	Migração, checksum e data de aplicação

As restrições incluem unicidade de e-mail, unicidade de CNPJ quando informado, vínculo composto e no máximo um `OWNER` por organização.

Event

A tabela `events` possui:

- `id`.
- `organization_id`.
- `status`.
- `payload`.
- `created_at`.

O payload JSONB contém os dados gerais, palestrantes e atividades.

As atividades não possuem uma tabela relacional independente no Event. Elas são elementos estruturados do evento.

Registration

Tabela	Dados principais
inventories	Atividade, evento e capacidade
registrations	Participante, atividade, chave idempotente, estado, valor, prazo e snapshot
outbox	Identificador, envelope da mensagem e data de publicação

O snapshot permite apresentar dados da atividade e do evento associados à inscrição sem consultar tabelas de outro domínio.

Payment

A tabela `payments` possui:

- Identificador da inscrição como chave primária.
- Identificador do usuário.
- Valor em centavos.
- Estado.

- Primeira decisão simulada.
- Data de criação.

A chave primária limita a uma transação por inscrição.

Notification

A tabela `deliveries` possui:

- Identificador da mensagem como chave primária.
- Envelope recebido.
- Estado da entrega simulada.
- Contador de tentativas.
- Data de atualização.

A unicidade do identificador permite deduplicar mensagens republicadas.

Relacionamentos físicos

Figura 4. Relacionamentos físicos locais dos bancos



Somente relacionamentos físicos locais; demais vínculos por APIs e eventos.

Esse diagrama mostra as chaves estrangeiras locais. Os vínculos lógicos com Event, Payment e Notification são realizados por APIs e mensagens.

Tipos e contratos

No frontend, os tipos principais são:

- `User`.
- `LoginResult`.

- `Session`.
- `Organization`.
- `OrganizationMember`.
- `Activity`.
- `Speaker`.
- `EventData`.
- `Registration`.
- `Payment`.
- `PlatformStatus`.

Os contratos estão concentrados em `frontend/src/types/` e `frontend/src/services/business.ts`.

15. Fluxos de execução

Fluxo principal

1. O usuário realiza login.
2. O frontend recebe o JWT e o prazo.
3. O participante acessa o catálogo.
4. Consulta o evento e suas atividades.
5. Solicita inscrição com chave idempotente.
6. Registration consulta os dados da atividade.
7. O banco decide a disponibilidade sob bloqueio.
8. A inscrição é confirmada, reservada ou colocada em espera.
9. Havendo pagamento, Payment persiste a decisão simulada.
10. Registration aplica o resultado.
11. A mudança de estado gera um evento na outbox.
12. RabbitMQ entrega a mensagem ao Notification.
13. O frontend apresenta o estado retornado pelos serviços.

Sequência de pagamento

Figura 5. Sequência de inscrição paga e notificação

1. Inscrição e reserva

1	Participante	Frontend	Solicitar inscrição
2	Frontend	Gateway	POST /registrations
3	Gateway	Registration	Encaminhar solicitação
4	Registration	Event	Consultar evento e atividade
5	Event	Registration	Retornar capacidade, preço e programação
6	Registration	Registration	Reservar vaga sob transação
7	Registration	Frontend	Retornar PENDING_PAYMENT e prazo

2. Pagamento e conciliação

8	Participante	Frontend	Aprovar a simulação
9	Frontend	Gateway	POST /payments/:id
10	Gateway	Payment	Encaminhar decisão
11	Payment	Payment	Persistir a decisão em PENDING
12	Payment	Registration	Aplicar o resultado do pagamento
13	Registration	Registration	Validar prazo, confirmar e registrar outbox
14	Registration	Payment	Retornar CONFIRMED
15	Payment	Payment	Registrar APPROVED
16	Payment	Frontend	Apresentar o resultado

3. Notificação assíncrona

17	Registration	RabbitMQ	Publicar o evento da outbox
18	RabbitMQ	Notification	Entregar a mensagem
19	Notification	Notification	Deduplicar e registrar a entrega simulada

A notificação ocorre após a confirmação e não bloqueia o resultado da inscrição.

Tratamento de falhas

Situação	Comportamento definido
Credencial inválida	Requisição recusada
Organização sem vínculo	Operação não autorizada
Atividade lotada	Entrada na lista de espera
Pagamento recusado	Inscrição cancelada e vaga liberada
Reserva vencida	Inscrição expirada
Aprovação tardia	Inscrição terminal não é reaberta
Callback indisponível	Payment mantém decisão para conciliação
RabbitMQ indisponível	Outbox permanece para publicação posterior
Notification indisponível	Confirmação da inscrição é preservada
Mensagem duplicada	Identificador impede duplicação do efeito simulado

16. Integrações externas

Cloudflare Tunnel

O túnel foi previsto para encaminhar o acesso externo ao Nginx local, evitando a contratação de uma VPS para a demonstração.

O endereço público depende da configuração do ambiente. Nenhuma URL pública específica é atribuída neste relatório.

Figma

O Figma foi utilizado como referência visual para login, cadastro, perfis, catálogo, gestão, criação de eventos e inscrição.

Os assets utilizados pela aplicação são locais. O funcionamento do site não depende dos links temporários de exportação do Figma.

Pagamentos

Não foi adotado um provedor financeiro externo. A aprovação e a recusa são simuladas, sem cobrança real.

Notificações e recuperação

As entregas são simuladas no ambiente do projeto. A adoção de SMTP ou outro provedor de comunicação representa uma evolução posterior.

17. Configuração e execução

Pré-requisitos

- Docker Desktop em modo Linux.
- Docker Compose.
- Node.js 24 para scripts e desenvolvimento local.
- Microsoft Edge para a configuração atual dos testes Playwright.
- Python compatível com os serviços para desenvolvimento fora dos containers.

O ambiente Docker utiliza Python 3.13 nos serviços Python.

Preparação

Na raiz, em PowerShell:

```
if (-not (Test-Path -LiteralPath '.env')) {
  Copy-Item -LiteralPath '.env.example' -Destination '.env'
}
node scripts/init-identity-keys.mjs
node scripts/init-internal-key.mjs
```

As configurações incluem:

- Porta HTTP.
- Credenciais dos cinco bancos.
- Credenciais do RabbitMQ.
- Chaves JWT.
- Credencial das chamadas internas.
- Prazo das reservas.

Os valores reais não devem ser versionados.

Inicialização

```
docker compose up --build -d --wait --wait-timeout 180
```

A aplicação local é acessada em: `http://localhost:8080`

Portas

Aplicação	Porta interna
Nginx	80
Frontend estático	80
Gateway	3000
Identity	3000
Payment	3000
Event	8000
Registration	8000
Notification	8000
PostgreSQL	5432
RabbitMQ	5672

Somente a porta HTTP do Nginx é publicada no host pela configuração principal.

Monitoramento e encerramento

```
docker compose ps
docker compose logs --tail 100
docker compose down
```

O encerramento normal preserva os volumes.

Desenvolvimento do frontend

```
npm --prefix frontend ci
npm --prefix frontend run dev
```

Build:

```
npm --prefix frontend run build
```

O proxy do Vite encaminha `/api` para o ambiente local na porta 8080.

Persistência e migrações

O script de infraestrutura cria bancos e permissões em um volume novo.

O Identity aplica migrações numeradas, com controle de checksum e bloqueio de inicialização concorrente. Os demais serviços inicializam suas tabelas no banco proprietário.

Alterar credenciais no arquivo de ambiente não altera automaticamente usuários já criados em um volume PostgreSQL existente.

18. Testes

A estratégia de validação foi organizada em testes unitários, de integração, de concorrência e de navegador.

Testes unitários

Cobrem principalmente:

- Hash e verificação de senha.
- Autenticação.
- Assinatura e validade dos tokens.
- Restrições de perfil.
- Permissões institucionais.
- Proteção do Gateway.
- Saúde das aplicações.

Comandos:

```
npm --prefix gateway test
npm --prefix services/identity test
npm --prefix services/payments test
```

Testes de integração

Arquivo	Finalidade
platform.test.mjs	Infraestrutura, saúde e isolamento dos bancos
identity.test.mjs	Cadastro, login e perfil
identity-migrations.test.mjs	Reaplicação das migrações
organizations.test.mjs	Vínculos e permissões
figma-identity.test.mjs	Campos de perfil e cadastro institucional
business.test.mjs	Eventos, inscrições, pagamento, fila e recuperação
notification-recovery.ps1	Indisponibilidade e recuperação do serviço

Testes de navegador

Os testes em `frontend/tests/` contemplam:

- Login e cadastros.
- Perfis.
- Termos e privacidade.
- Menu da conta e saída.
- Criação e publicação de evento.
- Catálogo e detalhes.

- Reserva e pagamento.
- Recuperação.
- Sessão de uma hora.
- Recarregamento e sincronização entre abas.
- Responsividade.

Execução:

```
npm --prefix frontend run test:e2e
```

Critérios de aceite da entrega

Cenário	Resultado exigido
Disputa pela última vaga	Não exceder capacidade
Solicitação repetida	Não duplicar a operação
Atividade gratuita	Confirmar quando houver vaga
Pagamento aprovado	Confirmar dentro do prazo
Pagamento recusado	Liberar vaga
Reserva expirada	Recusar aprovação tardia
Cancelamento	Atualizar ocupação e fila
Notification indisponível	Preservar confirmação
Mensagem repetida	Evitar duplicação do efeito
Acesso entre organizações	Recusar operação sem vínculo
Reload durante a sessão	Preservar login e prazo original
Saída da conta	Remover sessão persistida

A comprovação desses critérios deve acompanhar a entrega com registros das execuções e capturas pertinentes. Este relatório não introduz resultados numéricos de uma nova execução.

19. Estado final do desenvolvimento

No cenário de conclusão apresentado, a entrega contempla:

Componente	Situação no escopo
Frontend e navegação	Entregues
Identity e organizações	Entregues
Catálogo e gestão de eventos	Entregues

Componente	Situação no escopo
Programação e atividades	Entregues
Reserva e controle de vagas	Entregues
Lista de espera	Entregue
Pagamento simulado	Entregue
Notificação simulada	Entregue
Recuperação simulada	Entregue
Sessão persistente de uma hora	Entregue
Docker Compose e bancos próprios	Entregues
Contratos e ADRs	Documentados
Attendance	Modelado
Certificate	Modelado
Reporting	Modelado

O resultado é um protótipo acadêmico integrado. O encerramento desse recorte não equivale à implantação de uma plataforma comercial com alta disponibilidade, cobrança real ou certificação operacional.

20. Limitações e melhorias futuras

As limitações foram delimitadas para manter o projeto compatível com o contexto acadêmico.

Limitação do recorte	Evolução possível
Pagamento simulado	Integração com provedor financeiro
Comunicação simulada	Entrega por SMTP ou serviço especializado
JWT sem revogação centralizada	Gestão de sessões e invalidação de credenciais
Sessão em localStorage	Avaliação de cookie HttpOnly e proteção correspondente
PostgreSQL compartilhado	Separação física conforme necessidade
Capacidade fixa após publicação	Contrato coordenado de alteração
Processamento periódico	Workers com métricas e políticas de retentativa mais completas
Serviços modelados	Implementação de presença, certificados e relatórios
Execução local	Automação de implantação e operação contínua

Também são evoluções recomendadas:

- Automatização de integração contínua.
- Estratégia de backup e restauração.
- Ampliação dos testes de falha e carga.
- Monitoramento do backlog e das filas de falhas.
- Rastreabilidade de ponta a ponta.
- Paginação e filtros no servidor.
- Procedimentos de rotação de chaves e segredos.

Esses itens ampliam a maturidade operacional sem alterar os fundamentos da arquitetura.

21. Comparação entre planejamento e entrega

Planejamento original	Solução da entrega	Observações
Plataforma de eventos	Sistema com catálogo, programação e inscrições	Objetivo central preservado
Arquitetura de microsserviços	Cinco domínios implementados	Gateway separado
Duas tecnologias de backend	Node.js e Python	REST/JSON como contrato comum
Bancos por serviço	Cinco bancos lógicos com credenciais próprias	Instância física compartilhada
Autenticação JWT	JWT RS256	Chave privada restrita ao Identity
Perfis de participante e organizador	Conta pessoal e vínculos institucionais	Permissões por organização
Telas do Figma	Componentes e páginas adaptados	Dados reais e layout responsivo
Inscrições concorrentes	Transação e lock de inventário	Controle no banco
Pagamento simulado	Decisão persistida e conciliação	Sem integração financeira
Notificações assíncronas	Outbox, RabbitMQ e consumidor	Entrega simulada
Presença e certificados	Serviços modelados	Implementação posterior prevista
Relatórios	Domínio Reporting modelado	Projeções sem consultas cruzadas
Disponibilização externa	Arquitetura preparada para Tunnel	Endereço depende do ambiente

As principais alterações incorporadas durante o desenvolvimento foram a separação dos acessos, a senha mínima de oito caracteres, o cadastro institucional transacional e a sessão persistente de uma hora.

22. Encerramento e próximos passos recomendados

O EventHub encerra o escopo acadêmico com uma arquitetura que permite acompanhar uma operação desde a autenticação até a confirmação da inscrição e o processamento de sua notificação.

A divisão dos serviços preserva a propriedade dos dados e torna explícitas as decisões necessárias para manter consistência entre capacidade, reservas e pagamentos.

A continuidade recomendada é:

1. Consolidar a versão de entrega no controle de versão.
2. Organizar as evidências de testes e demonstração.
3. Validar a configuração de exposição pelo Cloudflare Tunnel.
4. Implementar Attendance, mantendo unicidade do check-in.
5. Implementar Certificate com elegibilidade baseada em presença e término.
6. Implementar Reporting com projeções próprias.
7. Evoluir segurança, observabilidade e operação conforme o uso real.

O produto final desse recorte reúne interface, serviços, persistência, comunicação e documentação suficientes para apresentar a proposta do EventHub, demonstrar os principais desafios de uma arquitetura distribuída e orientar sua evolução sem reconstruir o contexto do projeto.